Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование точности подсчёта значения функции в точке с помощью рядов Тейлора»

Выполнил:
студент группы
3825Б1ПМии1
Казарин А.М.

Проверила:
Бусько П.В.

Нижний Новгород
2025

Содержание:

Введение	3
Постановка задачи	4
Руководство пользователя	5
Описание программной реализации	6
Результаты экспериментов	8
Заключение	11
Литература	12
Приложение	13

Введение

Эта работа посвящена исследованию влияния порядка суммирования чисел в рядах Тейлора на точность вычисления различных функций. Эта работа актуальна, так как в программировании часто возникают проблемы из за округления больших чисел. С её помощью можно будет оценить в каких случаях обратное суммирование будет точнее прямого.

Постановка задачи

Цель работы: исследовать влияние прямого и обратного порядка суммирования членов ряда Тейлора на точность вычисления значений функций sin(x), cos(x), exp(x) и ln(1+x).

1. Реализовать на языке C алгоритмы вычисления значений функций с помощью рядов Тейлора, используя рекуррентные формулы, структуры для хранения промежуточных данных и указатели на функции.

2. Провести сравнительный анализ погрешностей вычислений для прямого и обратного порядков суммирования при различных значениях аргумента и количества членов ряда.

3. Определить условия, при которых обратный порядок суммирования дает более точный результат и построить график для выбранного случая.

Руководство пользователя

Пользователь должен запустить программу, и он увидит в интерфейсе предложение выбрать функции и пронумерованный список функций. Затем он выбирает одну из 4 функций: sin(x), cos(x), exp(x), ln(x+1) нажатием соответствующей цифры на клавиатуре и нажимает Enter. Далее ему предлагается ввести значение x, это будет точка в которой будет считаться значение для выбранной функции. Потом пользователь вводит произвольное число элементов в ряде тейлора (желательно 15-20). После счёта каждого элемента будет учитываться ошибка, постоянно сравнивая с эталонным значением. Больше никаких действий от пользователя не требуется и он может наблюдать результаты эксперимента в зависимости от введенных им изначальных данных. Результаты представлены в табличном формате для удобства чтения и проверки *(Рис. 1)*. В таблице представлены значения ошибок для прямой и обратной сумм. В правом столбце находится эталонное значение из библиотеки math.h. В последней строке выводится какой подсчет оказался более точным, прямой или обратный.

```
Консоль отладки Microsoft Visual Studio
Choose your fucntion!
 1: sin(x)
 2: cos(x)
 3: exp(x)
 4: ln(x+1)
4
Set value for x (float)
0.99
Set number of members (int)
15
Number    Direct sum       Reversed sum     Expected
1         3.02e-01         2.76e-02         6.88e-01
2         1.88e-01         9.62e-01         6.88e-01
3         1.35e-01         4.72e-01         6.88e-01
4         1.05e-01         7.96e-01         6.88e-01
5         8.53e-02         5.56e-01         6.88e-01
6         7.16e-02         7.46e-01         6.88e-01
7         6.15e-02         5.89e-01         6.88e-01
8         5.38e-02         7.22e-01         6.88e-01
9         4.77e-02         6.07e-01         6.88e-01
10        4.27e-02         7.08e-01         6.88e-01
11        3.87e-02         6.18e-01         6.88e-01
12        3.52e-02         6.99e-01         6.88e-01
13        3.23e-02         6.25e-01         6.88e-01
14        2.98e-02         6.93e-01         6.88e-01
15        2.76e-02         6.31e-01         6.88e-01
Reversed sum is more accurate
```

Рис. 1 Пример работы программы.

Описание программной реализации

В моём проекте 1 основной файл: Taylor.cpp .— это основной файл, в нём содержатся только директивы и функция `main ()` *(Приложение 1)*. В нём я в начале инициализирую структуру `xn` что под именем `data`. Далее вывожу на экран с помощью `printf()` изначальный интерфейс для выбора функции, точки и кол-ва членов в ряду и соответственно считываю значения, которые вводит пользователь с помощью `scanf_s()`. Затем я обращаюсь к массиву указателей на мои функции, чтобы вызвать одну из них по выбору пользователя `functions[choice - 1](&data, x, n).` Также прописываю отдельный случай для логарифма, так как там x+1. И ещё вызываю эталонное значение, уже обращаясь к массиву указателей на функции из библиотеки `math.h`. Затем уже вывожу в табличном формате значение ошибки и эталонное, а после сравнивая последнее значение в массиве ошибки при прямом суммировании и первое значение из массива обратного суммирования (последний шаг), вывожу какой подсчет оказался точнее.

Перед мейном, в файле содержится следующий список функций и структур:

`struct xn -` структура члена ряда тейлора. В ней содержится кол-во элементов, массив значений, массив ошибки: прямой и обратный, массив частичных сумм: прямой и обратной.

`void my_sin(struct xn* data, float x, int n)` - принимает в себя структуру `data`, точку для подсчёта и кол-во членов. в самой функции производится подсчёт функции синуса с помощью рекуррентной формулы и двух циклов for для прямого и обратного суммирования, подсчитывая на каждой итерации значения ошибки и занося их в соответствующие массивы. Дальнейшие функции различаются лишь рекуррентными формулами. Для синуса: $a_0 = x$, $a_{(n+1)} = a_n * (-x^2) / ((2n+2)(2n+3))$

`void my_cos(struct xn* data, float x, int n) -` принимает в себя структуру `data`, точку для подсчёта и кол-во членов. в самой функции производится подсчёт функции косинуса с помощью рекуррентной формулы и двух циклов for для прямого и обратного суммирования, подсчитывая на каждой итерации значения

ошибки и занося их в соответствующие массивы. Для косинуса: a_0 = 1, a_(n+1) = a_n * (-x²) / ((2n+1)*(2n+2))

void my_exp(struct xn* data, float x, int n) - принимает в себя структуру data, точку для подсчёта и кол-во членов. в самой функции производится подсчёт функции экспоненты с помощью рекуррентной формулы и двух циклов for для прямого и обратного суммирования, подсчитывая на каждой итерации значения ошибки и занося их в соответствующие массивы. Для экспоненты: a_0 = 1, a_(n+1) = a_n * (x / (n+1))

void my_log(struct xn* data, float x, int n) - принимает в себя структуру data, точку для подсчёта и кол-во членов. в самой функции производится подсчёт функции натурального логарифма с помощью рекуррентной формулы и двух циклов for для прямого и обратного суммирования, подсчитывая на каждой итерации значения ошибки и занося их в соответствующие массивы. Для логарифма: a_1 = x, a_n = -a_(n-1) * x * (n-1)/n

void (*functions[4])(struct xn*, float, int) = { my_sin, my_cos, my_exp, my_log } - Массив указателей на функции, в которых я произвожу прямой и обратный подсчет, с помощью цикла for. Массив состоит из 4 элементов и принимает в себя структуру точки, соответственное значение самой точки x, и кол-во элементов.

float (*etalon[4])(float) = { sin, cos, exp, log } - Массив указателей на функции из библиотеки math.h. Принимает в качестве аргументов, значение точки x, для которой и производим подсчет. В дальнейшем будет использоваться для присваивания эталонного значения.

Результаты экспериментов

В качестве эксперимента я взял функции ln(x+1), sin(x), cos(x), exp(x) и считал их значения в точке x=0.99, выбрал первые 15 членов в ряду Тейлора. Далее будет представлены графики (рис. 2,3,4,5) на которых будут наглядно представлены значения ошибки во время счёта прямой и обратной сумм на каждом из 15 шагов.

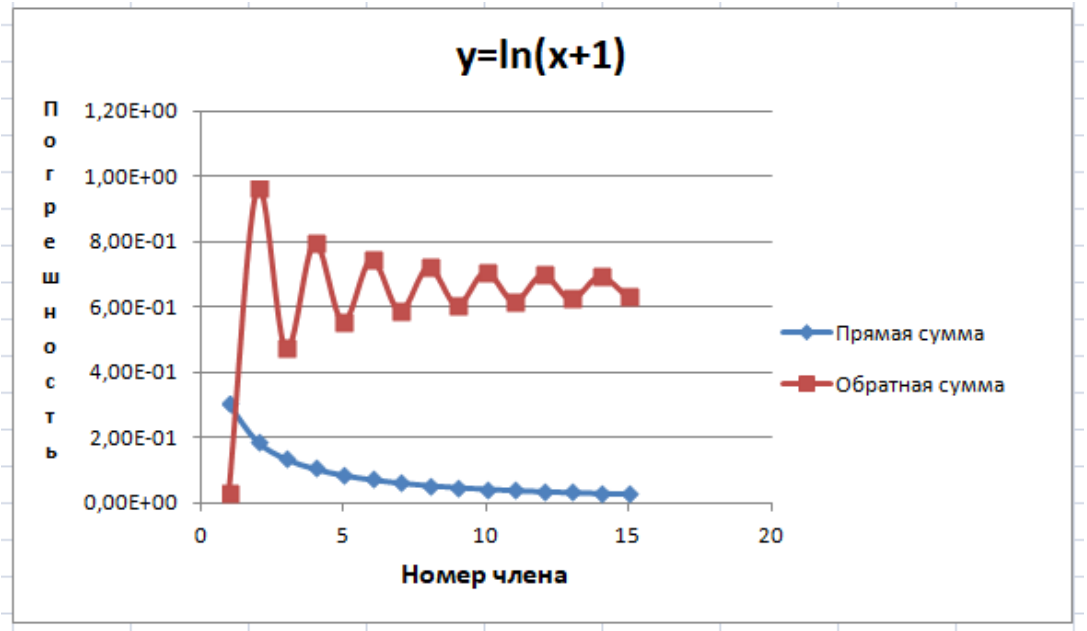

Рис. 2 Подсчет ошибки на каждом шаге ln(x+1).

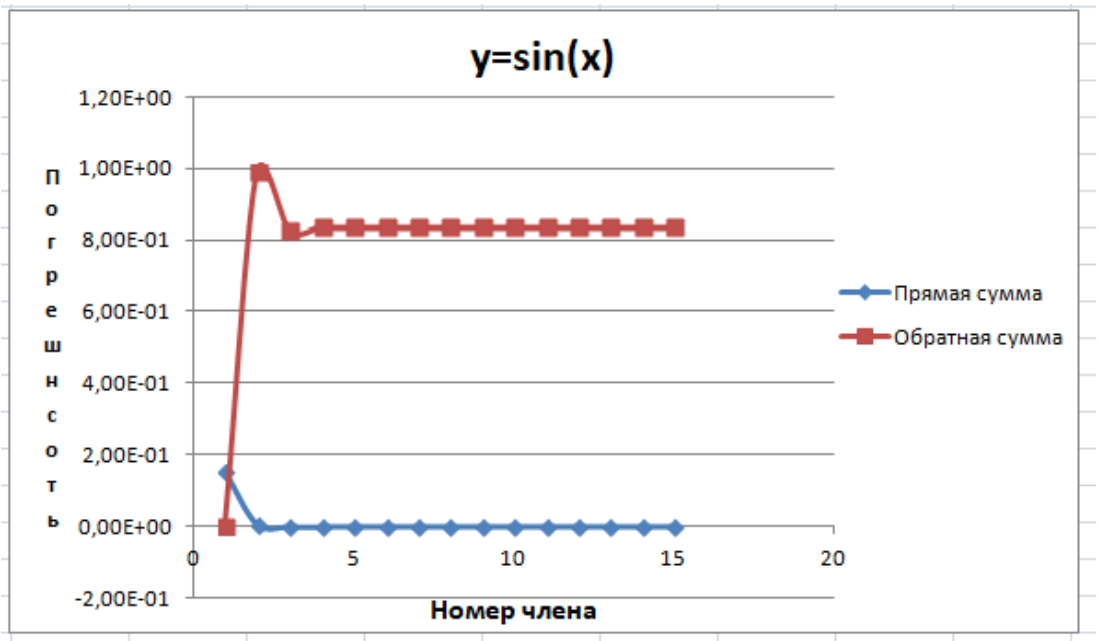

Рис. 3 Подсчет ошибки на каждом шаге sin(x).

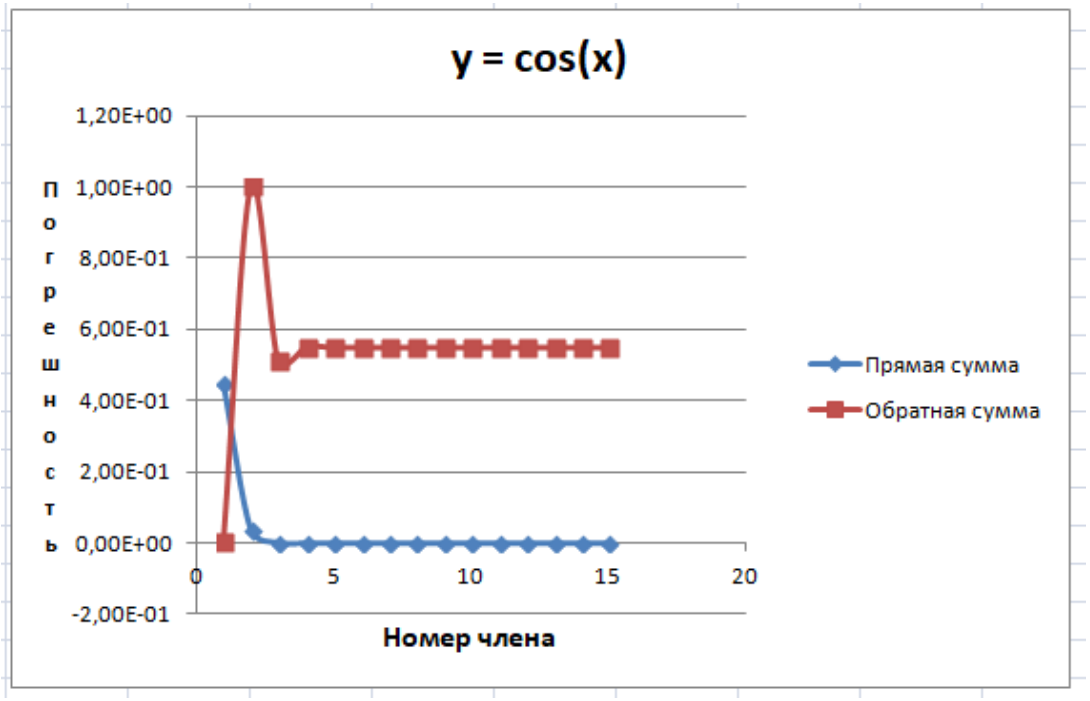

Рис. 4 Подсчет ошибки на каждом шаге cos(x).

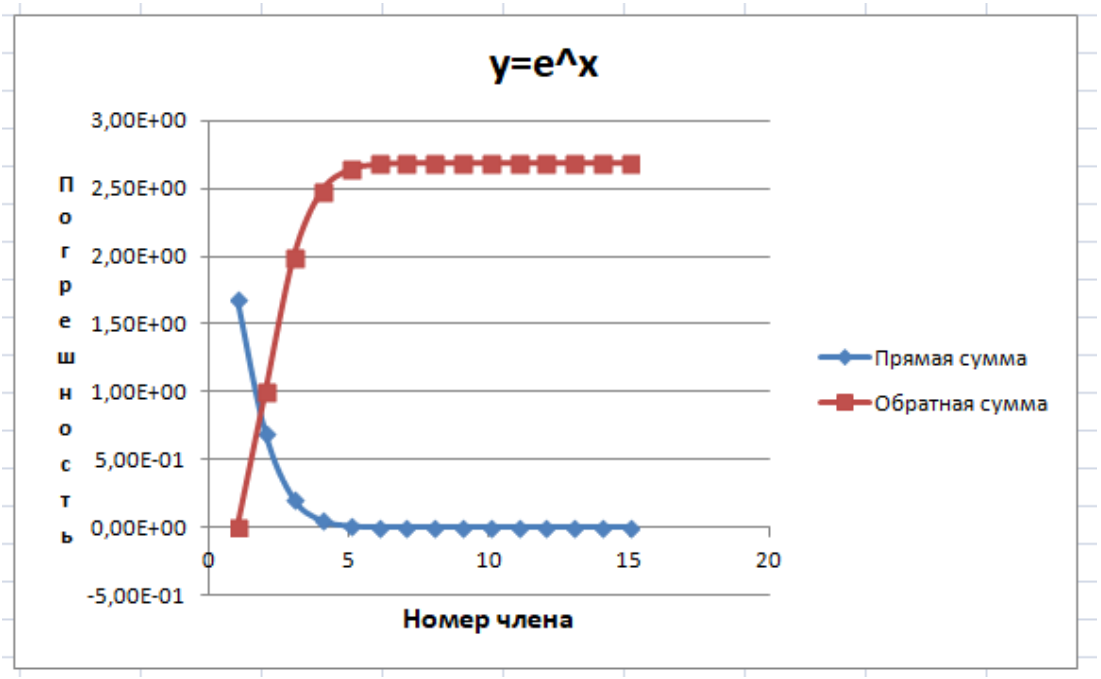

Рис. 5 Подсчет ошибки на каждом шаге e^x.

Графики прямой суммы строятся с начала, а графики обратной суммы с конца, поэтому будут сравниваться, первый элемент красных графиков и последний элемент синих (это все значения ошибок при разных суммированиях). Как можно заметить,

элементы красного графика расположены чуть ниже синего на графиках натурального логарифма и экспоненты, соответственно обратное суммирование было точнее. Но это не всегда верно, всё зависит от кол-ва членов и насколько быстро ряд начинает сходится. Чем больше членов ряда учитывается, тем точнее результат, но при этом растет вычислительная погрешность, особенно при прямом суммировании, где накопление ошибки происходит быстрее.

Заключение

В ходе выполнения лабораторной работы я написал программу для вычисления функций с использованием рядов Тейлора с прямым и обратным порядком суммирования. Экспериментально подтверждено, что для знакопеременных рядов с медленно убывающими членами, как например ln(1+x), обратный порядок суммирования позволяет снизить вычислительную погрешность в несколько раз по сравнению с прямым. Полученные результаты могут быть использованы при разработке численных алгоритмов, требующих повышенной точности вычислений.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение

1. Функция `main()`

```c
int main()
{
    struct xn data;

    printf("Choose your fucntion!\n 1: sin(x)\n 2: cos(x)\n 3: exp(x)\n 4: ln(x+1)\n");
    int choice;
    scanf_s("%d", &choice);

    printf("Set value for x (float)\n");
    float x;
    scanf_s("%f", &x);

    printf("Set number of members (int)\n");
    int n;
    scanf_s("%d", &n);

    functions[choice - 1](&data, x, n);

    float exact;
    if (choice == 4)
        exact = etalon[3](x + 1);
    else
        exact = etalon[choice - 1](x);


    printf("Number\t Direct sum\t Reversed sum\t Expected\n");
    for (int i = 0; i < n; i++)
        printf("%d\t%.2e\t%.2e\t%.2e\n", i + 1, data.err[i], data.err_rev[i], exact);

    if (data.err[n - 1] > data.err_rev[0])
        printf("Reversed sum is more accurate\n");
    else
        printf("Direct sum is more accurate\n");


    free(data.x); free(data.err); free(data.err_rev); free(data.sum); free(data.sum_rev);
}
```